

Graphs in C

Frans Ekberg

Spring 2025

Introduction

A graph is a data structure that is comprised of a set of nodes. These nodes are then linked to each other using connections. These connections can be strung together to make a paths between different nodes. Connections can go only one way or both ways between two nodes. In this report, all connections are two-way connections. The goal of this report is to construct a program that can find the shortest path between two cities in a data set of Swedish train routes.

Benchmark

To set up a benchmark, the **time.h** library is used using the following code:

```
clock_gettime(CLOCK_MONOTONIC, &t_start);  
...  
clock_gettime(CLOCK_MONOTONIC, &t_stop);
```

The code starts a clock, runs the code that need to be evaluated and then stops the clock. The time is then recorded and stored. The benchmarks in this report will take a relatively long time compared to the clocks frequency. This means that one measurement will suffice to get an accurate execution time.

Creating the map

To create a graph or map of the given data, some structures need to be set up. The data is a CSV-file of direct rail connection between 52 cities in Sweden. Each row in the file contains two cities and the time it takes to travel between them. For example, the first line in the file looks like this:

Stockholm,Södertälje,21

To be able to use the data some structures need to be set up.

Firstly, a structure for a city is constructed. A city struct contains the name of the city, an id set by the order the cities are read in, and an linked list of the cities connections, which is the second data structure. A connection struct contains a pointer to the origin city, a pointer to the destination city, and the time it takes to travel between the cities. The last structure is the map. The map struct contains an array of all cities and a variable for the number of cities.

The program will have to access cities many, many times as it searches for the shortest path between two cities. To use as little time as possible when trying to access a city, a hash map can be used. The hash key can be obtained by converting the name of a city into an integer and then hashing that integer. This results in a very short access time for each city. The hash table is the array that is stored in the map struct.

Each row in the file is read into a buffer, then the string is divided into it's three parts and the following code is run:

```
city *src = lookup(trains, strtok(copy, ","));
city *dst = lookup(trains, strtok(NULL, ","));
unsigned int dist = atoi(strtok(NULL, ","));

connect(src, dst, dist);
```

Taking the example from earlier (*Stockholm,Södertälje,21*) and working through the code with it will look like this:

The **lookup** function looks in the hash table if it can find a city matching the name *Stockholm*. If it doesn't exist, it creates a new city named *Stockholm* and returns a pointer to it, or it finds it and returns the pointer. It does the same for *Södertälje*. Then the distance between them is read. These three variables are sent to the **connect** function. It makes one connection for each city and puts them in the city's respective connections list. This means that *Stockholm* has a connection like this: *Stockholm-Södertälje, 21* After going the whole CSV-file, the maps is done, with all the cities and their connections.

The naive search function

To find the shortest path between two cities, a **search** function is made. It receives an **origin city (from)** and a **destination city (to)** and a **limit**. The limit is a number that sets the limit for the longest path the function can try to find. If the function finds a path that is longer than the limit, it cannot continue searching on that path. This is in place to prevent infinite loops, as the search algorithm can go back and forth between two cities forever anyways. This is what makes this function **naive**, it cannot detect loops.

The function works by recursively calling itself on the next city in the current city's connections. Then it checks if the destination city has been reached. If it has, it returns 0, as the distance between a city and itself is 0. If it hasn't found the destination city it simply returns the distance between the current city and the next one. When going back up the recursion the distances are added together and the smallest total distance is returned. If the distance reaches above the limit, the function doesn't call itself and instead returns -1. This is not added to the total at the end, so the return value of -1 is essentially the function saying, "don't continue searching here".

Doing some benchmarks for different origin and destinations gives the results following results, see *Table 1* below.

From	To	Limit	Path	Time (<i>ms</i>)
Malmö	Göteborg	200	153	0.02
Malmö	Göteborg	300	153	0.54
Göteborg	Stockholm	300	211	0.44
Malmö	Stockholm	300	273	0.55
Stockholm	Sundsvall	400	327	12.36
Göteborg	Sundsvall	550	515	5353.95
Stockholm	Umeå	600	517	23738.15
Stockholm	Umeå	550	517	3616.72
Sundsvall	Umeå	500	190	0.05
Sundsvall	Umeå	300	190	0.00
Umeå	Göteborg	800	705	1.60
Göteborg	Umeå	710	Unkn	DNF

Table 1: Execution time of finding the shortest path of some cities using the **naive** search function

Most connections can be found, but some take a long time. The connection between Göteborg and Umeå took too long for the program to find, so the program was cut short. The DNF stands for Did Not Finish and the path should be 705 but, as stated, the program didn't get to finish. It is interesting

to note that the same path can take a dramatically different amount of time to find depending on the limit imposed on the program. This is of course because the algorithm can look at longer paths when a larger limit is imposed.

To fix make the algorithm able to find long paths in a reasonable amount of time, the first improvement that can be made is to be able to detect loops.

Loop detection

Instead of having a limit for the longest path that can be found, the search algorithm can be made to detect loops. To do this, the algorithm need some way to know which cities it have already been to. This can be done by using an array of cities called a **path**. This array will contain all the cities that have already been visited, and if the algorithm tries to go to a city where it have already been, it will stop itself.

To implement this in the current code, the search function will also receive an array called **path**. Before recursively calling itself on the next city in the connection it checks if that city is present in the path array. If it's not present, the function hasn't been here yet, but it is going here now, so it adds the city to the array. If the city is already in the array, it detects a loop. Instead of the code about having a limit, if at any point a loop is detected, the function doesn't call itself and instead returns -1. This means that any path that is about to loop is thrown away, and only the paths without loops are counted.

Doing some benchmarks for different origin and destinations gives the results following results, see *Table 2* below.

From	To	Path	Time (ms)
Malmö	Göteborg	153	517.51
Malmö	Helsingborg	48	1485.99
Stockholm	Göteborg	211	1471.28
Göteborg	Stockholm	211	1482.89
Umeå	Göteborg	705	869.60
Göteborg	Umeå	705	3364.86
Sundsvall	Umeå	190	2015.69
Umeå	Sundsvall	190	773.28
Malmö	Kiruna	1162	2344.36
Kiruna	Malmö	1162	2624.58

Table 2: Execution time of finding the shortest path of some cities using the **noLoops** search function

A noticeable improvement can be seen, as the route Göteborg-Umeå can be found in a pretty quick compared to the results in *Table 1*. Even the longest route on the map, Malmö-Kiruna, can be found. However, almost all short routes now take a longer time to find. This is because the paths can be any length, and thus, even when a short path is found, all other paths are explored, even the ones longer than the shortest path currently found. This is why a route like Malmö-Helsingborg takes a long time, even though they are only two stations apart. A simple way to improve the current performance is by introducing the limit again, but instead of setting it at the start, it is set when a path to the destination is found.

Dynamic maximum distance

To implement this, the search function now receives an additional variable for the **maximum distance** of the path. At first it is set to null, as no path to the destination have been found yet. However, as soon as a path to the destination is found, the search function now sets the maximum distance to the length of that path. This ensures that the function doesn't try to find paths that are longer than the current shortest path.

Doing the same benchmarks as in **Table 2** gives the results following results, see *Table 3* below.

From	To	Path	Time (ms)
Malmö	Göteborg	153	0.18
Malmö	Helsingborg	48	0.63
Stockholm	Göteborg	211	0.10
Göteborg	Stockholm	211	0.12
Umeå	Göteborg	705	0.35
Göteborg	Umeå	705	71.70
Sundsvall	Umeå	190	1590.55
Umeå	Sundsvall	190	0.00
Malmö	Kiruna	1162	706.46
Kiruna	Malmö	1162	0.80

Table 3: Execution time of finding the shortest path of some cities using the **improved noLoop** search function

The overall performance is greatly improved as can be seen by comparing *Table 2* and *Table 3*. One interesting thing that was found out while trying to figure out why the Sundsvall-Umeå time was so much larger than the Umeå-Sundsvall time is that the order of the cities in the CVS file matters. If the Sundsvall-Umeå connection happens to be before all other connections that include either city, aka the first connection from Sundsvall becomes Umeå,

then its almost instant, just like Umeå-Sundsvall. This is to be expected of course, but still interesting how much time difference it can make. The long execution time of the route seen in *Table 3*, is cause by the algorithm having to search the whole network south of Sundsvall before getting to the Umeå connection.

Conclusion

Searching in a graph can be pretty complicated. Because all nodes lead to each other, many checks have to be preformed, to get a decent execution time. Otherwise endless loops can occur, or the path that is searched becomes longer than a path that is already found. To be able to search for paths faster or in lager graphs, improvements would need to bee made. One such improvement could be a more complicated, but faster algorithm, maybe using breath-first search instead of the current deep-first search method.